

Feature Extraction

Robots in Context

Leon Bodenhagen lebo@mmmi.sdu.dk

Associate Professor

SDU Robotics, Maersk Mc-Kinney Moller Institute



Content

- Feature extraction in laser scans
 - Features
 - Line Fitting
 - Line Extraction Algorithms
- Introduction to Localization



Feature detection – why?

- Determine relation to environment
- Two basic approaches
 - From raw data
 - From higher level structures



Figure 4.34

The perceptual pipeline: from sensor readings to knowledge models.



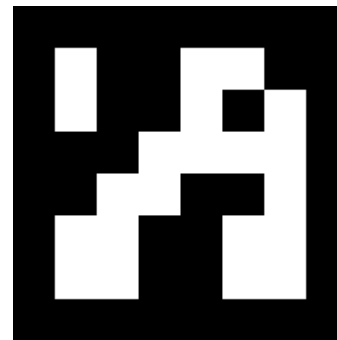
Feature detection – what?

- Good features are easily detected and robust
- Abstraction trade-off
 - Low level – lines, circles, polygons..
 - High level – doors, tables..
- Markers



Feature detection – what?

- Good features are easily detected and robust
- Abstraction trade-off
 - Low level – lines, circles, polygons..
 - High level – doors, tables..
- Markers

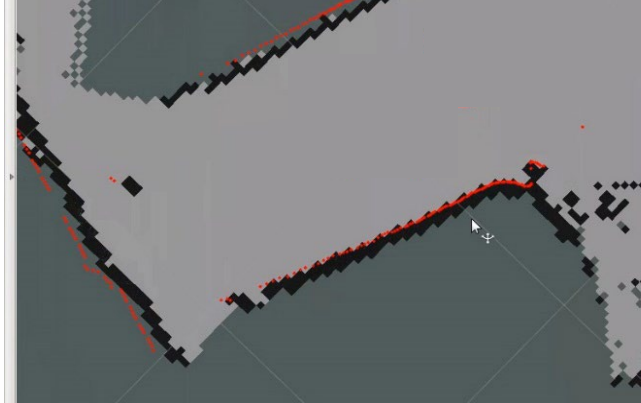


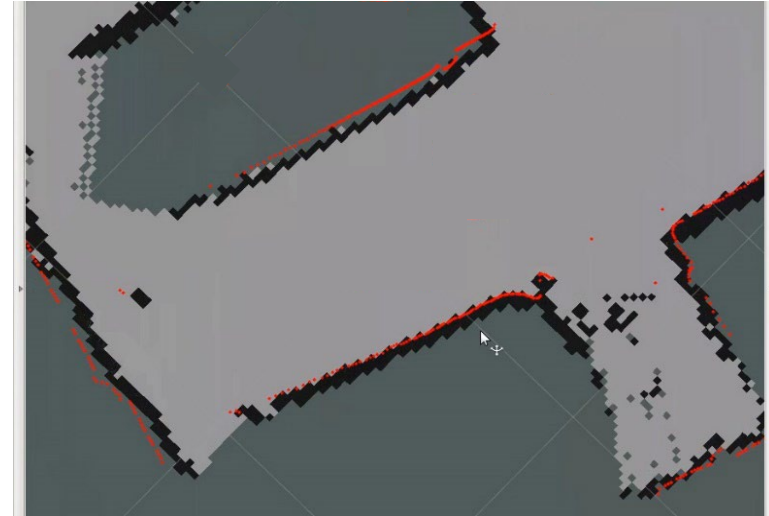
We

-



Line Feature Extraction

- Input
 - A scan, or a list of points
 - What is needed
 - Ability to determine which points belong to a line
 - Ability to fit a line to the points.
- 
- A grayscale image showing a scan of a document. A red line is drawn across the image, following the contours of a dark, irregular shape. The background is light gray, and the dark shape is black. The red line is composed of small red segments, likely representing a fitted line or a path through the points of the dark shape.



Line Representations

- Representation of a line:

$$x \cos(\alpha) + y \sin(\alpha) = r$$

- α is the angle between x-axis and the normal
 - r is the distance from the origin to the closest point on the line
- Why not use $y=ax+b$?



Line Fitting

- A given point (x, y) can be expressed as

$$x = \rho \cos(\theta), y = \rho \sin(\theta)$$

- The line consists of all points where

$$x \cos(\alpha) + y \sin(\alpha) = r$$

- For x, y to be on the line:

$$\rho \cos(\theta) \cos(\alpha) + \rho \sin(\theta) \sin(\alpha) = r$$

$$\Leftrightarrow \rho \cos(\theta - \alpha) - r = 0$$



Least Square Line fitting

Assuming there is noise on the measurements

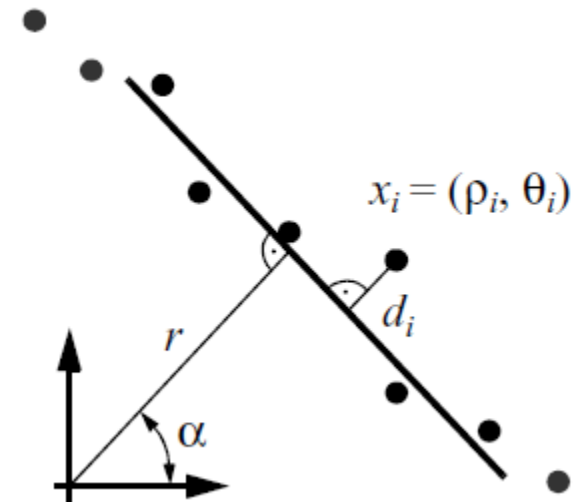
$$\rho \cos(\theta - \alpha) - r = d$$

The least square fit over N

measurements: $\min_{\alpha, r} S = \min_{\alpha, r} \sum_i^N d_i^2$

We can include a weight in case not all points should have similar importance

$$S = \sum_i^N w_i d_i^2$$



Least Square Line fitting

- Objective $S = \sum w_i d_i^2 = \sum w_i (\rho_i \cos(\theta_i - \alpha) - r)^2$.
- Optimum where?



Least Square Line fitting

- Objective $S = \sum w_i d_i^2 = \sum w_i (\rho_i \cos(\theta_i - \alpha) - r)^2 .$

- Optimum where $\frac{\partial S}{\partial \alpha} = 0 \quad \frac{\partial S}{\partial r} = 0 .$

- Solution $\alpha = \frac{1}{2} \text{atan} \left(\frac{\sum w_i \rho_i^2 \sin 2\theta_i - \frac{2}{\sum w_i} \sum \sum w_i w_j \rho_i \rho_j \cos \theta_i \sin \theta_j}{\sum w_i \rho_i^2 \cos 2\theta_i - \frac{1}{\sum w_i} \sum \sum w_i w_j \rho_i \rho_j \cos(\theta_i + \theta_j)} \right)$

$$r = \frac{\sum w_i \rho_i \cos(\theta_i - \alpha)}{\sum w_i}$$

Least Square Line Fitting - Pitfalls

- The book assumes the best fit when

$$\frac{\partial S}{\partial \alpha} = 0 \quad \frac{\partial S}{\partial r} = 0.$$

- This can also be the worst fit, hence the line perpendicular.
- To fix it
 - Set $\alpha = \alpha + \frac{\pi}{2}$ and recompute r .
- How to detect it
 - Option 1: Check the higher order derivative to see if it is a local minimum or maximum.
 - Option 2: Compute the residual for both cases and choose the one with the smallest residual.
 - Option 3: Use prior knowledge about the expected direction of the line
- Other pitfall
 - The algorithm may generate negative values of r . In this case set $\alpha = \alpha + \pi$ and change the sign of r .



Line Fitting

- We now know the least square solution. Do this mean that we are done?



Line Fitting

- We now know the least square solution. Does this mean that we are done?
 - No. We still do not know which points are to be associated with the line.



Line Extraction Algorithms

- Algorithm from paper
 - Split-and-merge
 - Incremental
 - RANSAC
 - Hough transform
- Will not consider
 - Line regression
 - Expectation-Maximization



Split and merge

Algorithm 1: *Split-and-Merge*

- 1 Initial: set s_1 consists of N points. Put s_1 in a list $\mathcal{L}$
 - 2 Fit a line to the next set s_i in $\mathcal{L}$
 - 3 Detect point P with maximum distance d_P to the line
 - 4 If d_P is less than a threshold, continue (go to 2)
 - 5 Otherwise, split s_i at P into s_{i1} and s_{i2} , replace s_i in $\mathcal{L}$ by s_{i1} and s_{i2} , continue (go to 2)
 - 6 When all sets (segments) in $\mathcal{L}$ have been checked, merge collinear segments.
-



Incremental

Algorithm 3: *Incremental*

- 1 Start by the first 2 points, construct a line
 - 2 Add the next point to the current line model
 - 3 Recompute the line parameters
 - 4 If it satisfies line condition, continue (go to 2)
 - 5 Otherwise, put back the last point, recompute the line parameters, return the line
 - 6 Continue with the next 2 points, go to 2
-



RANSAC

Algorithm 4: *RANSAC*

- 1 Initial: A set of N points
 - 2 **repeat**
 - 3 Choose a sample of 2 points uniformly at random
 - 4 Fit a line through the 2 points
 - 5 Compute the distances of other points to the line
 - 6 Construct the inlier set
 - 7 If there are enough inliers, recompute the line parameters, store the line, remove the inliers from the set
 - 8 **until** *Max.N.Iterations reached or too few points left*
-



Hough Transform

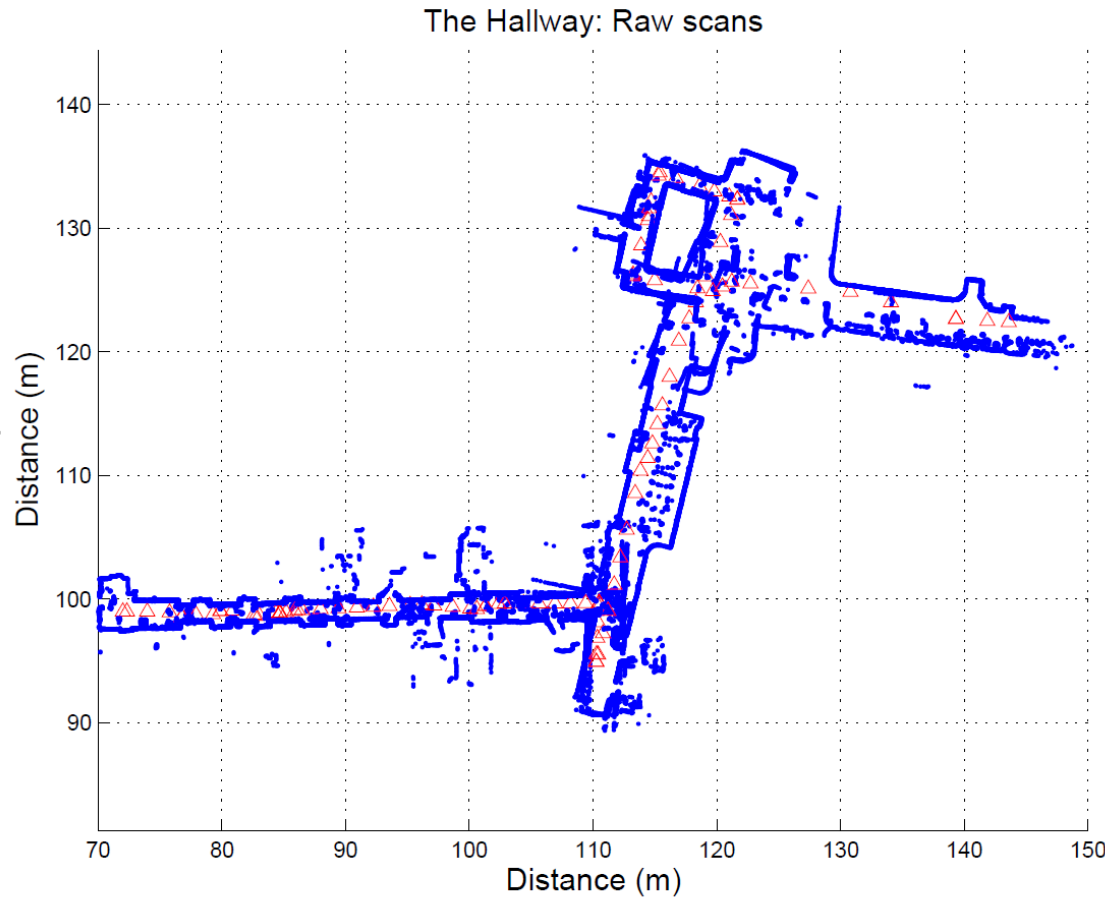
Algorithm 5: *Hough-Transform*

- 1 Initial: A set of N points
- 2 Initialize the accumulator array (*model space*)
- 3 Construct values for the array
- 4 Choose the element with max. votes V_{max}
- 5 If V_{max} is less than a threshold, terminate
- 6 Otherwise, determine the inliers
- 7 Fit a line through the inliers and store the line
- 8 Remove the inliers from the set, goto 2



Comparison

- Environment mapped by robot.
- Clustering based on discontinuities.



Comparison

EXPERIMENTAL RESULTS OF THE ALGORITHMS

Algorithm	Complexity	Speed [Hz]	N.Lines	Correctness		Precision	
				TruePos [%]	FalsePos [%]	$\sigma_{\Delta r}$ [cm]	$\sigma_{\Delta \alpha}$ [deg]
Split-Merge + Clus.	$N \times \log N$	1470	641	86.0	8.9	1.95	0.74
Incremental	$S \times N^2$	344	561	77.8	5.9	2.04	0.72
Incremental + Clus.		617	567	79.2	5.1	2.04	0.76
Line Regression	$N \times N_f$	364	577	76.4	10.1	1.99	0.80
LR + Clus.		384	562	75.8	8.4	1.97	0.79
RANSAC	$S \times N \times N.Trials$	29	749	75.6	31.5	1.68	0.77
RANSAC + Clus.		93	547	70.7	12.2	1.37	0.70
Hough Transform	$S \times N \times NC + S \times NR \times NC$	8	825	82.0	32.5	1.63	0.76
HT + Clus.		9	600	79.5	10.0	1.51	0.67
EM	$S \times N1 \times N2 \times N$	0.6	1153	78.6	53.7	2.09	0.97
EM + Clus.		0.7	709	80.3	23.1	1.58	0.73

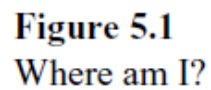


Comparison

EXPERIMENTAL RESULTS OF THE ALGORITHMS

Algorithm	Complexity	Speed [Hz]	N.Lines	Correctness		Precision	
				TruePos [%]	FalsePos [%]	$\sigma_{\Delta r}$ [cm]	$\sigma_{\Delta \alpha}$ [deg]
Split-Merge + Clus.	$N \times \log N$	1470	641	86.0	8.9	1.95	0.74
Incremental	$S \times N^2$	344	561	77.8	5.9	2.04	0.72
Incremental + Clus.		617	567	79.2	5.1	2.04	0.76
Line Regression	$N \times N_f$	364	577	76.4	10.1	1.99	0.80
LR + Clus.		384	562	75.8	8.4	1.97	0.79
RANSAC	$S \times N \times N.Trials$	29	749	75.6	31.5	1.68	0.77
RANSAC + Clus.		93	547	70.7	12.2	1.37	0.70
Hough Transform	$S \times N \times NC + S \times NR \times NC$	8	825	82.0	32.5	1.63	0.76
HT + Clus.		9	600	79.5	10.0	1.51	0.67
EM	$S \times N1 \times N2 \times N$	0.6	1153	78.6	53.7	2.09	0.97
EM + Clus.		0.7	709	80.3	23.1	1.58	0.73



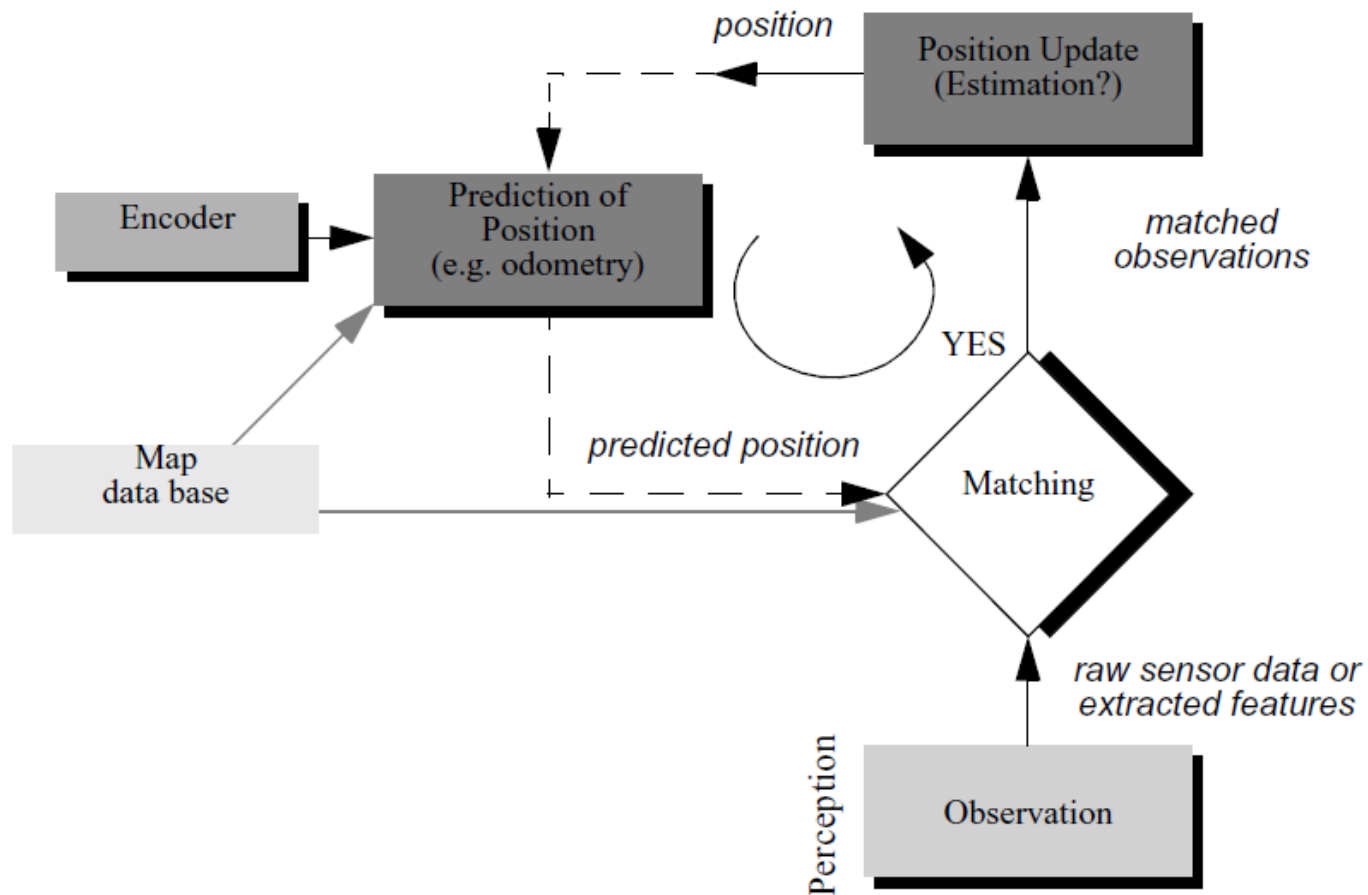
[illegible]

Localization

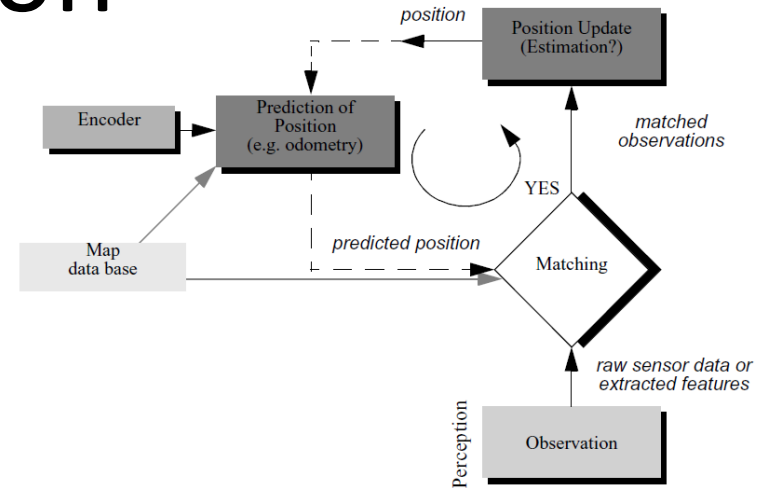
- Two cases
 - When we have a map of the environment (Localization)
 - When we need to construct the map as we go along (Simultaneous Localization And Map building (SLAM)).



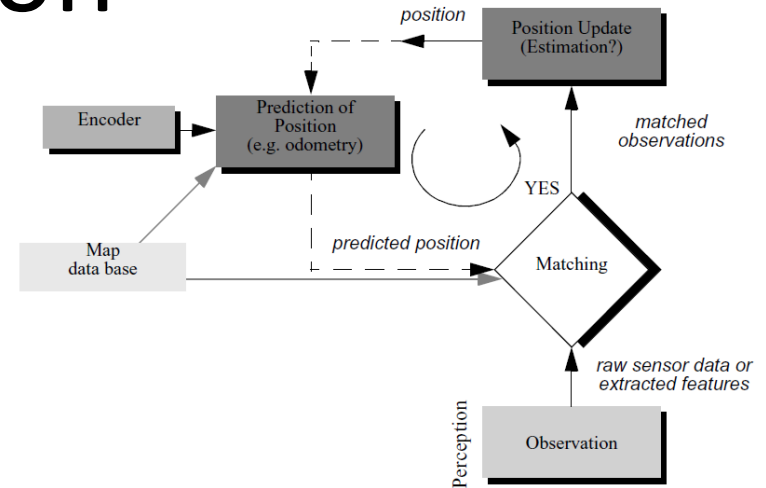
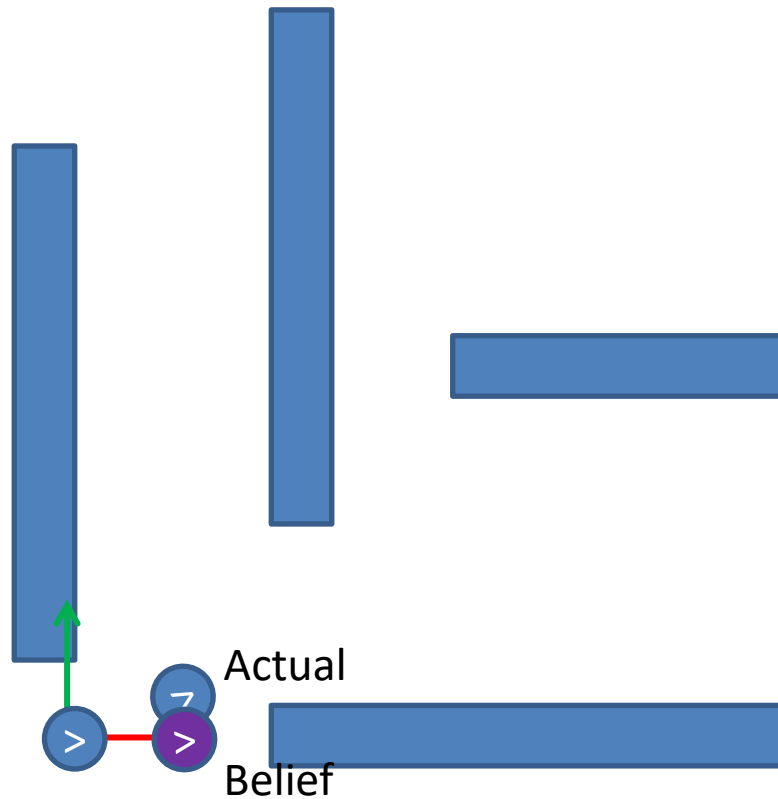
Localization



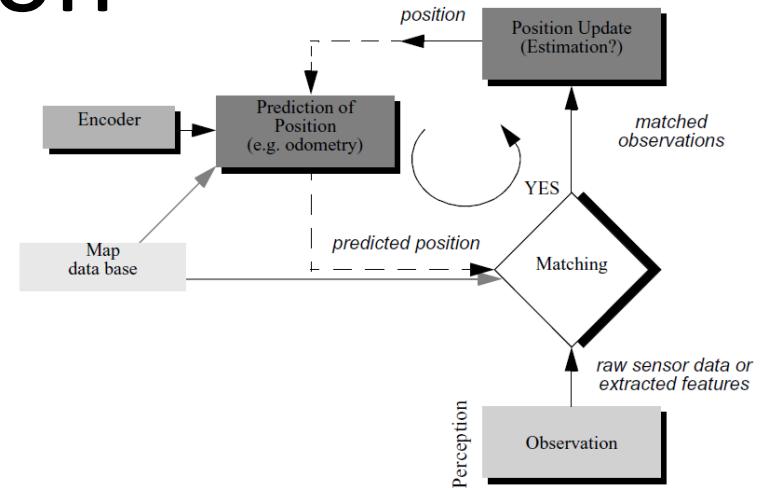
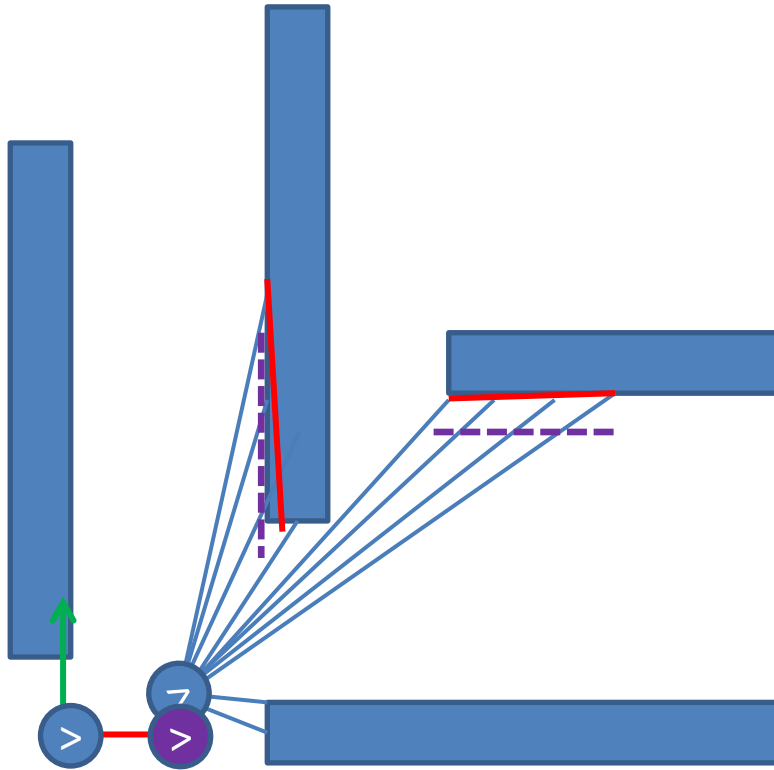
A diagram showing a robot (a blue circle with a white greater-than sign '>') at the bottom left. A green arrow points upwards from the robot, and a red arrow points to the right. The environment contains several blue rectangular obstacles: a tall one on the left, a tall one in the upper center, a horizontal one in the upper right, and a long horizontal one at the bottom right.

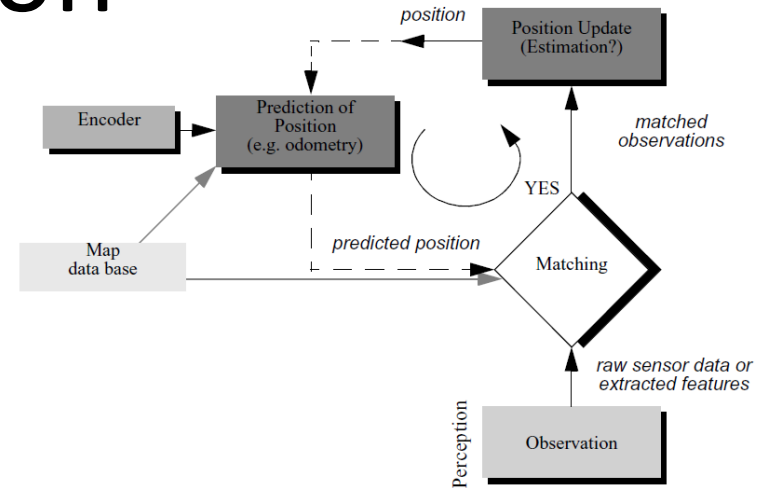


Localization



Localization





Localization

- Two general challenges – noise and aliasing
 - Sensor noise – what to trust
 - More involved features such as lines are more robust than single points.
 - Sensor aliasing – non-unique sensor readings
 - The more complex the feature, the more unique a matching/aliasing we can make
 - Effector noise – odometry errors, accumulate



Summary

- Feature extraction
 - Features
 - Line Representation and Fitting
 - Line Extraction Algorithms
- Introduction to Localization

